

QA Architecture Lab — Complete Build & CI/CD Playbook

Jenkins · Docker · k3d · Kubernetes · REST Assured

Repository	github.com/kanhaya/qa-architecture-lab
Pipeline	qa-k3d-pipeline @ http://localhost:8080
Audience	Senior SDETs & QA Architects
Status	End-to-end pipeline verified 

This document is a step-by-step record of **what was built**, **why each decision was made**, **what was achieved**, and **how to reproduce the full stack** from scratch.

PDF version: Run `./scripts/generate-playbook-pdf.sh` to regenerate `docs/QA-Architecture-Lab-Playbook.pdf`

Table of Contents

- [1. Overall Goal](#)
- [2. What We Achieved](#)
- [3. End-to-End Architecture](#)
- [4. Phase 1 — Loan Service \(Spring Boot\)](#)
- [5. Phase 2 — Unit Tests](#)
- [6. Phase 3 — Docker Containerization](#)
- [7. Phase 4 — Kubernetes \(k3d\) Deployment](#)

8. [Phase 5 — API Automation \(REST Assured\)](#)
 9. [Phase 6 — Deployment Scripts](#)
 10. [Phase 7 — Jenkins CI/CD Pipeline](#)
 11. [Port Map Reference](#)
 12. [Problems Solved & Lessons Learned](#)
 13. [Diagnostic Command Sheet](#)
 14. [Verification Checklist](#)
 15. [How to Explain This Like a Pro](#)
 16. [Reproduce Everything From Scratch](#)
 17. [Verify a Successful Build](#)
 18. [Production Maturity Path](#)
-

1. Overall Goal

Build a **production-style QA Architecture Lab** that demonstrates the full lifecycle of a cloud-native microservice:

Layer	Purpose
Application	A real REST API (Loan Service) that SDETs can test
Unit tests	Fast feedback on business logic
Docker	Package the app as a portable container image
Kubernetes (k3d)	Run 3 replicas with health probes and a NodePort service
API automation	REST Assured smoke/functional/negative/regression suites
CI/CD (Jenkins)	Automate build → image → deploy → test on every commit

Target audience: Senior SDETs and QA Architects learning how modern teams ship and validate microservices.

Repository: github.com/kanhaya/qa-architecture-lab

2. What We Achieved

By the end of this lab you have:

-  A working **Loan Management REST API** with CRUD operations
 -  **Unit tests** for service, repository, controller, and exception handling
 -  A **multi-stage Dockerfile** with a built-in health check
 -  **Kubernetes manifests** (namespace, configmap, deployment, service)
 -  A **k3d cluster** with an embedded Docker registry and NodePort access
 -  **REST Assured API tests** (smoke, functional, negative, regression)
 -  **Shell scripts** for cluster setup, deploy, validate, and dashboard
 -  A **Jenkins pipeline** that builds, pushes, deploys, and runs smoke tests automatically
 -  A **successful end-to-end build** from GitHub → Jenkins → k3d → passing tests
-

3. End-to-End Architecture



▼
JUnit / Surefire
Jenkins Report

Application internal architecture

HTTP Request



LoanController → LoanService → LoanRepository → InMemoryLoanRepository
(ConcurrentHashMap)

4. Phase 1 — Loan Service (Spring Boot)

What we built

A Maven multi-module project with a `loan-service` module:

```
loan-service/
├── src/main/java/com/qa/loanservice/
│   ├── controller/      LoanController.java
│   ├── service/         LoanService.java
│   ├── repository/      InMemoryLoanRepository.java
│   ├── model/           Loan, LoanStatus, DTOs
│   └── exception/       GlobalExceptionHandler, LoanNotFoundException
└── src/main/resources/
    ├── application.yml   (local - port 8081)
    ├── application-docker.yml (Docker - port 8080)
    └── application-kubernetes.yml (k8s - port 8080)
```

API endpoints

Method	Path	Description
GET	/api/loans	List all loans
GET	/api/loans/{id}	Get loan by ID
POST	/api/loans	Create loan (status: PENDING)
PUT	/api/loans/{id}/status	Update loan status
DELETE	/api/loans/{id}	Delete loan
GET	/actuator/health	Health check (used by k8s probes)
GET	/actuator/info	Application info

Seeded test data

ID	Customer	Amount	Status
101	Rahul	500,000	APPROVED
102	Amit	300,000	PENDING

Why port 8081 locally?

Jenkins runs on **port 8080**. To avoid conflicts during local development, the app defaults to **8081** on the host:

```
# application.yml
server:
  port: ${SERVER_PORT:8081}
```

Inside Docker and Kubernetes containers the app still listens on **8080**.

How to run locally

```
cd loan-service
mvn spring-boot:run

curl http://localhost:8081/actuator/health
curl http://localhost:8081/api/loans
```

5. Phase 2 — Unit Tests

What we built

JUnit 5 + Mockito tests inside `loan-service/src/test/` :

Test class	What it covers
<code>LoanServiceTest</code>	Business logic, create/update/delete
<code>InMemoryLoanRepositoryTest</code>	Data storage, seeded data
<code>LoanControllerTest</code>	HTTP layer with MockMvc
<code>GlobalExceptionHandlerTest</code>	Error responses (400, 404)

Why separate from API tests?

Unit tests run **without a running server** — fast feedback in CI (`mvn test -pl loan-service`). API tests (Phase 5) require a deployed service.

How to run

```
mvn test -pl loan-service
```

6. Phase 3 — Docker Containerization

What we built

A **multi-stage Dockerfile** at the project root:

Stage	Base image	Purpose
Build	<code>eclipse-temurin:21-jdk-alpine</code>	Compile with Maven
Runtime	<code>eclipse-temurin:21-jre-alpine</code>	Run JAR as non-root user

Key design choices:

- **Non-root user** (`appuser`) for security
- **curl** installed for Docker `HEALTHCHECK`
- **Health check** hits `/actuator/health` every 30s

- Container listens on **port 8080** (`APP_ENV=docker` profile)

Why multi-stage?

Keeps the final image small (~200 MB vs ~600 MB with JDK) and follows production best practices.

How to build and run

```
mvn clean package -pl loan-service -am -DskipTests
docker build -t loan-service:1.0 .
docker run -p 8081:8080 -e APP_ENV=docker loan-service:1.0

curl http://localhost:8081/actuator/health
```

7. Phase 4 — Kubernetes (k3d) Deployment

What is k3d?

k3d runs a lightweight Kubernetes cluster (k3s) inside Docker containers on your local machine. It is ideal for learning and CI/CD labs without needing a cloud account.

What we built

Kubernetes manifests in `k8s/` :

File	Purpose
<code>namespace.yaml</code>	Isolates resources in <code>qa-lab</code> namespace
<code>configmap.yaml</code>	Sets <code>APP_ENV=kubernetes</code> , <code>SERVER_PORT=8080</code>
<code>deployment.yaml</code>	3 replicas, health probes, resource limits
<code>service.yaml</code>	NodePort 30080 → pod port 8080

Cluster setup script

`scripts/setup-k3d-cluster.sh` creates:

Component	Value
Cluster name	qa-lab (default) or qa-cluster (your setup)
Registry (host)	localhost:5001
Registry (in-cluster)	k3d-qa-registry:5000
NodePort	30080
Namespace	qa-lab

Why two registry URLs?

k3d exposes **one physical registry** under two names:

```

Your Mac / Docker daemon      Kubernetes pods
    |                          |
localhost:5001 —same registry— k3d-qa-registry:5000
(docker push)                (image pull)

```

Deployment highlights

```

# deployment.yaml
replicas: 3
readinessProbe: GET /actuator/health (port 8080)
livenessProbe:  GET /actuator/health (port 8080)
image: k3d-qa-registry:5000/loan-service:1.0

```

How to deploy manually

```

./scripts/setup-k3d-cluster.sh

docker build -t localhost:5001/loan-service:1.0 .
docker push localhost:5001/loan-service:1.0

kubectl apply -f k8s/
kubectl set image deployment/loan-service \
  loan-service=k3d-qa-registry:5000/loan-service:1.0 -n qa-lab

kubectl get pods -n qa-lab
curl http://localhost:30080/actuator/health
curl http://localhost:30080/api/loans

```

8. Phase 5 — API Automation (REST Assured)

What we built

A separate Maven module `tests/` with REST Assured + JUnit 5:

```
tests/src/test/java/com/qa/tests/
├── base/           BaseApiTest.java
├── clients/        LoanClient.java
├── models/         LoanResponse.java
├── utils/          TestConfig.java, TestDataFactory.java
├── tests/
│   ├── smoke/      SmokeTests.java      @Tag("smoke")
│   ├── functional/ FunctionalTests.java  @Tag("functional")
│   ├── negative/   NegativeTests.java    @Tag("negative")
│   └── regression/ RegressionSuite.java  (all tags)
```

Test URL auto-detection

`TestConfig.java` probes these URLs in order:

1. `http://localhost:8081` — local Spring Boot
2. `http://localhost:30080` — Kubernetes NodePort
3. Falls back to `8081` if nothing responds

It validates the response contains `"status":"UP"` so Jenkins on port 8080 is never mistaken for the Loan Service.

How to run

```
# Against local app
export BASE_URL=http://localhost:8081
mvn test -pl tests -Dgroups=smoke

# Against Kubernetes
export BASE_URL=http://localhost:30080
mvn test -pl tests -Dgroups=smoke
```

9. Phase 6 — Deployment Scripts

Script	Purpose
<code>setup-k3d-cluster.sh</code>	Create k3d cluster, registry, apply manifests
<code>deploy-and-test.sh</code>	Full flow: build → push → deploy → validate → smoke tests
<code>validate-deployment.sh</code>	Poll health endpoint, check pod count
<code>start-lab.sh</code>	Start k3d + Kubernetes Dashboard
<code>setup-k8s-dashboard.sh</code>	Install dashboard, print access token
<code>open-dashboard.sh</code>	Port-forward dashboard to https://localhost:8443
<code>setup-jenkins-k3d-network.sh</code>	Connect Jenkins container to k3d Docker network
<code>configure-jenkins-kubeconfig.sh</code>	Generate kubeconfig inside Jenkins (used by pipeline)
<code>export-jenkins-kubeconfig.sh</code>	Export kubeconfig for manual debugging

One-command deploy and test

```
./scripts/deploy-and-test.sh
```

10. Phase 7 — Jenkins CI/CD Pipeline

Jenkins setup

Jenkins runs as a **Docker container** with:

- Port **8080** exposed (UI at <http://localhost:8080>)
- Docker socket mounted (`-v /var/run/docker.sock:/var/run/docker.sock`)
- Connected to the **k3d Docker network** (`k3d-qa-cluster`)

```
# One-time: connect Jenkins to k3d network
JENKINS_CONTAINER=jenkins ./scripts/setup-jenkins-k3d-network.sh
```

Pipeline job

- **Job name:** `qa-k3d-pipeline`

- **URL:** <http://localhost:8080/job/qa-k3d-pipeline/>
- **Source:** GitHub repo `kanhaya/qa-architecture-lab` (branch `main`)
- **Definition:** `Jenkinsfile` in repo root

Pipeline stages (in order)

#	Stage	What it does	Why
1	Checkout	Pulls code from GitHub	Always start from latest commit
2	Verify Environment	Checks docker, kubectl, mvn, registry, k3d API	Fail fast if infrastructure is broken
3	Build Application	<code>mvn clean package -DskipTests</code>	Compile JAR without waiting for API tests
4	Build Docker Image	<code>docker build + docker push</code>	Package app, push to k3d registry
5	Deploy to Kubernetes	<code>kubectl apply + kubectl set image</code>	Roll out new image to 3 pods
6	Wait for Application	<code>kubectl rollout status</code>	Ensure pods are healthy before testing
7	Run REST Assured Tests	<code>mvn test -pl tests -Dgroups=smoke</code>	Validate deployed API automatically

Image tagging strategy

Each build tags the image with the Jenkins `BUILD_NUMBER` :

```
Build:   k3d-qa-registry:5000/loan-service:27
Push:    localhost:5001/loan-service:27      (same registry, host port)
Deploy:  kubectl set image ... loan-service:27
```

Key environment variables in Jenkinsfile

```
REGISTRY      = 'k3d-qa-registry:5000'    // in-cluster image name
HOST_REGISTRY = 'localhost:5001'          // docker push target
BASE_URL      = 'http://k3d-qa-cluster-server-0:30080' // API tests from Jenkins
NAMESPACE    = 'qa-lab'
K3D_CLUSTER   = 'qa-cluster'
```

Kubeconfig inside Jenkins

The pipeline generates kubeconfig at runtime via `scripts/configure-jenkins-kubeconfig.sh` :

1. Runs `k3d kubeconfig get qa-cluster` using the k3d Docker image
2. Rewrites API server to `https://k3d-qa-cluster-serverlb:6443`
3. Uses embedded TLS certificates (no manual credential upload needed)

Shell steps use `#!/usr/bin/env bash` because Jenkins defaults to `/bin/sh` (dash) which does not support `source` .

11. Port Map Reference

Service	Port	Who uses it
Jenkins UI	8080	You (browser) — not the Loan Service
Loan Service (local dev)	8081	<code>mvn spring-boot:run</code> on your Mac
Loan Service (container/k8s)	8080	Inside Docker pod
Loan Service (k8s NodePort)	30080	You + Jenkins API tests from Mac
k3d registry (host push)	5001	<code>docker push</code> from Jenkins/host
k3d registry (in-cluster)	5000	Kubernetes image pull
Kubernetes Dashboard	8443	https://localhost:8443
k3d API server (from Jenkins)	6443	<code>kubectl</code> inside Jenkins container

Quick health checks

```
# Loan Service on Kubernetes
curl http://localhost:30080/actuator/health

# Loan Service local dev
curl http://localhost:8081/actuator/health

# k3d registry
curl http://localhost:5001/v2/_catalog

# Jenkins (should NOT be used for API tests)
curl http://localhost:8080/
```

12. Problems Solved & Lessons Learned

Problem 1: Port 8080 conflict (Jenkins vs Loan Service)

Symptom: API tests returned `403 Forbidden` with `X-Jenkins` headers.

Cause: Tests pointed at `localhost:8080` which is Jenkins, not the Loan Service.

Fix: Local app uses port **8081**; Kubernetes tests use **30080**; Jenkins pipeline uses `k3d-qa-cluster-server-0:30080` .

Problem 2: `k3d-qa-registry:5000` — no such host

Symptom: `docker push k3d-qa-registry:5000` failed with DNS lookup error.

Cause: That hostname only exists on the k3d Docker network, not on the host where the Docker daemon runs.

Fix: Tag image as `k3d-qa-registry:5000/...` but push via `localhost:5001` (same physical registry).

Problem 3: kubeconfig TLS error inside Jenkins

Symptom: `x509: certificate signed by unknown authority`

Cause: Host kubeconfig uses `https://0.0.0.0:<port>` and/or file-path certificates that don't exist inside Jenkins.

Fix: Generate kubeconfig at runtime with `configure-jenkins-kubeconfig.sh` , pointing API server to `k3d-qa-cluster-serverlb:6443` .

Problem 4: `source: not found` in Jenkins

Symptom: `source: not found` when running kubeconfig script.

Cause: Jenkins `sh` step uses `/bin/sh` (dash), not bash.

Fix: Add `#!/usr/bin/env bash` as the first line of shell blocks that use `source` .

Problem 5: Cluster name mismatch

Symptom: Wrong hostnames in pipeline (`k3d-qa-lab` vs actual `qa-cluster`).

Fix: Aligned `K3D_CLUSTER` , `K3D_NETWORK` , `BASE_URL` , and `KUBE_SERVER` to match actual cluster name `qa-cluster` .

Problem 6: `mvn: not found` in Jenkins

Symptom: Pipeline fails at Maven stage with `mvn: not found` .

Cause: Jenkins container does not have Maven installed or not in PATH.

Fix: Install Maven inside Jenkins (or use a custom Jenkins image). Verify with `docker exec jenkins mvn --version` .

Problem 7: `kubectl: not found` in Jenkins

Symptom: Deploy stage fails — `kubectl` command not found.

Cause: Base Jenkins image does not include `kubectl`.

Fix: Install `kubectl` in the Jenkins container. Verify with `docker exec jenkins kubectl version --client` .

Problem 8: Namespace `qa-lab` not found

Symptom: `kubectl apply` fails — namespace does not exist.

Cause: Namespaced resources applied before namespace exists (race when applying all manifests at once).

Fix: Apply `k8s/namespace.yaml` first, or use `kubectl apply -f k8s/` which creates namespace before other resources. Re-run apply if race occurs.

Problem 9: REST Assured `Connection refused`

Symptom: Tests fail with connection refused before deploy completes.

Cause: Functional tests ran during `mvn clean test` before the API was deployed.

Fix: Use `mvn clean package -DskipTests` in build stage. Deploy first, wait for rollout, then run `mvn test -pl tests`.

Key lesson: Maven builds the software. Kubernetes runs it. REST Assured validates the **running** system.

Problem 10: HTTP response to HTTPS client (ImagePullBackOff)

Symptom: Pods stuck in `ImagePullBackOff`; events show `http: server gave HTTP response to HTTPS client`.

Cause: Registry serves plain HTTP but containerd tries HTTPS.

Fix: Create registry as part of k3d cluster setup:

```
k3d registry create qa-registry --port 5001
k3d cluster create qa-cluster --registry-use k3d-qa-registry:5000
```

Diagnostic: `kubectl describe pod <pod-name> -n qa-lab` — read the Events section.

Problem 11: Image tag mismatch

Symptom: Kubernetes pulls `loan-service:1.0` but Jenkins built `loan-service:27`.

Cause: Deployment manifest has a static tag; pipeline builds with `${BUILD_NUMBER}`.

Fix: `kubectl set image deployment/loan-service loan-service=k3d-qa-registry:5000/loan-service:${BUILD_NUMBER} -n qa-lab`

Diagnostic:

```
kubectl get pod -n qa-lab -o jsonpath='{.items[0].spec.containers[0].image}'
```

Problem 12: Registry `localhost:5001` timeout from Jenkins

Symptom: `curl localhost:5001` fails inside Jenkins container.

Cause: Inside Jenkins, `localhost` means the Jenkins container itself — not the Mac host.

Fix: For HTTP checks from Jenkins, use `k3d-qa-registry:5000`. For `docker push` via socket, use `localhost:5001` (Docker daemon on host).

13. Diagnostic Command Sheet

When something fails, identify **which layer** failed, then run diagnostics for that layer only.

Layer 1 — Docker / Jenkins

```
docker ps
docker inspect jenkins
docker logs jenkins --tail 50
docker exec jenkins docker --version
docker exec jenkins mvn --version
docker exec jenkins kubectl version --client
```

Layer 2 — Network / Registry

```
docker network inspect k3d-qa-cluster
docker exec jenkins getent hosts k3d-qa-registry
docker exec jenkins curl -sf http://k3d-qa-registry:5000/v2/_catalog
curl -sf http://localhost:5001/v2/_catalog
curl -sf http://localhost:5001/v2/loan-service/tags/list
```

Layer 3 — Kubernetes

```
kubectl get nodes
kubectl get ns
kubectl get all -n qa-lab
kubectl get pods -n qa-lab -o wide
kubectl describe pod <pod-name> -n qa-lab # Most valuable for image failure
kubectl logs <pod-name> -n qa-lab
kubectl rollout status deployment/loan-service -n qa-lab
kubectl get svc loan-service -n qa-lab
```

Professional debugging pattern

```
Failure
↓
Identify the layer (build / registry / k8s / tests)
↓
Run the most direct diagnostic command
↓
Validate the hypothesis
↓
Change one thing
↓
Re-run pipeline
↓
Verify the next layer
```

14. Verification Checklist

Use this checklist every time you rebuild or debug the lab.

#	Check	Command	Expected result
1	Git repository	<code>git remote -v</code>	Points to <code>qa-architecture-lab</code>
2	Jenkins SCM	Build #N console	Checkout succeeds
3	Maven	<code>mvn --version</code> in Jenkins	Maven 3.9+
4	Docker	<code>docker --version</code> in Jenkins	Docker 20+
5	kubectl	<code>kubectl get nodes</code>	Node Ready
6	k3d API	<code>source scripts/configure-jenkins-kubeconfig.sh && kubectl get nodes</code>	Cluster reachable
7	Registry DNS	<code>docker exec jenkins getent hosts k3d-qa-registry</code>	Returns IP
8	Registry API	<code>curl http://k3d-qa-registry:5000/v2/_catalog</code> from Jenkins	JSON catalog
9	Image push	<code>curl localhost:5001/v2/loan-service/tags/list</code>	Contains build tag
10	Namespace	<code>kubectl get ns qa-lab</code>	Exists
11	Deployment	<code>kubectl rollout status deployment/loan-service -n qa-lab</code>	Successfully rolled out
12	Pods	<code>kubectl get pods -n qa-lab</code>	3/3 Ready
13	Service	<code>kubectl get svc -n qa-lab</code>	NodePort 30080
14	Health	<code>curl http://localhost:30080/actuator/health</code>	<code>{"status": "UP"}</code>
15	API	<code>curl http://localhost:30080/api/loans</code>	JSON with loans 101, 102
16	REST Assured	Jenkins test stage	Smoke tests PASS
17	Reports	Jenkins build page	Surefire/JUnit results visible

15. How to Explain This Like a Pro

15.1 Sixty-second interview answer

"I built a local CI/CD environment where Jenkins runs in Docker and orchestrates the complete test-deployment lifecycle. Jenkins checks out a Maven project from GitHub, builds the application, packages it into a Docker image tagged with the Jenkins build number, and pushes it to a local k3d registry. Kubernetes deploys that exact image through a Deployment with three replicas and exposes it through a NodePort Service. The pipeline waits for rollout readiness before executing REST Assured smoke tests. Results are published back to Jenkins. I intentionally separated application build from environment-dependent API tests because the latter require a live deployed service."

15.2 Common interview questions

Q: Why not run `mvn test` first?

These are black-box API tests against a running service. Running them during the initial Maven build creates a hidden environment dependency. I build and package first, deploy the exact image, wait for readiness, then execute REST Assured.

Q: Why Kubernetes Service?

Pods are ephemeral — their IPs change. The Service provides a stable network endpoint and load-balances traffic across ready pods.

Q: Why a registry?

The Docker image Jenkins builds must be available to the Kubernetes runtime. The registry is the contract between image creation and cluster deployment.

Q: Why build-number tags?

They make deployments traceable. I can identify exactly which Jenkins build produced the image running in Kubernetes. It avoids ambiguity from a mutable `latest` tag.

Q: How did you debug ImagePullBackOff?

I used `kubectl describe pod` and read the Events section. The event showed `HTTP response to HTTPS client`, which identified a registry protocol mismatch — not an application failure. I then verified registry reachability and reconfigured `k3d` registry integration.

Q: Why port 8081 locally but 30080 in Kubernetes?

Jenkins occupies port 8080 on my Mac. Local dev uses 8081 to avoid conflict. In Kubernetes, the Service exposes the app on NodePort 30080 while pods listen on 8080 internally.

16. Reproduce Everything From Scratch

Prerequisites

```
# Install on macOS  
brew install java maven docker k3d kubectl
```

Step 1 — Clone the repo

```
git clone https://github.com/kanhaya/qa-architecture-lab.git  
cd qa-architecture-lab
```

Step 2 — Run unit tests

```
mvn test -pl loan-service
```

Step 3 — Run locally

```
cd loan-service && mvn spring-boot:run
# In another terminal:
curl http://localhost:8081/api/loans
```

Step 4 — Create k3d cluster

```
./scripts/setup-k3d-cluster.sh
# Or if using qa-cluster name:
K3D_CLUSTER=qa-cluster k3d cluster create qa-cluster \
  --registry-create qa-registry:0.0.0.0:5001 \
  --port 30080:30080@server:0
kubectl apply -f k8s/
```

Step 5 — Deploy to Kubernetes

```
./scripts/deploy-and-test.sh
```

Step 6 — Start Jenkins

```
docker run -d \
  --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts

# Connect to k3d network
JENKINS_CONTAINER=jenkins ./scripts/setup-jenkins-k3d-network.sh
```

Install Maven, Docker CLI, and kubectl inside Jenkins (or use a custom Jenkins image).

Step 7 — Create Jenkins pipeline job

1. Open <http://localhost:8080>
2. New Item → Pipeline → name: `qa-k3d-pipeline`
3. Pipeline → Definition: **Pipeline script from SCM**
4. SCM: Git → URL: `https://github.com/kanhaya/qa-architecture-lab.git`
5. Branch: `main` → Script Path: `Jenkinsfile`
6. Save → **Build Now**

Step 8 — Verify successful build

See [Section 17](#).

17. Verify a Successful Build

In Jenkins

1. Open <http://localhost:8080/job/qa-k3d-pipeline/>
2. Latest build should show **green** (Success)
3. Check stages: all 7 stages completed
4. Console output should end with smoke tests passing

On your Mac

```
# Pods running (3 replicas)
kubectl get pods -n qa-lab

# Service reachable
curl http://localhost:30080/actuator/health
# Expected: {"status":"UP"}

curl http://localhost:30080/api/loans
# Expected: JSON array with loans 101 and 102

# Image in registry
curl http://localhost:5001/v2/_catalog
# Expected: {"repositories":["loan-service"]}
```

Expected pod output

NAME	READY	STATUS	RESTARTS	AGE
loan-service-xxxxxxxx-xxxxx	1/1	Running	0	2m
loan-service-xxxxxxxx-xxxxx	1/1	Running	0	2m
loan-service-xxxxxxxx-xxxxx	1/1	Running	0	2m

18. Production Maturity Path

The local lab is intentionally simple. The same architecture evolves into a production-grade platform:

Local lab	Production evolution
k3d local registry	GHCR, ECR, GCR, or enterprise registry with TLS
Build-number tags	Immutable image digests for provenance
Single Jenkins pipeline	Separate build, publish, and deploy jobs
Smoke tests only	Smoke → functional → regression stages
No security scanning	SAST, SCA, container scanning
Static k8s YAML	Helm or Kustomize per environment
Manual cluster	Ephemeral namespaces per PR/build
No observability	Prometheus, Grafana, centralized logs

Maturity ladder

```
Local learning lab
  ↓
CI pipeline (Jenkins)
  ↓
Container registry
  ↓
Kubernetes deployment
  ↓
Automated API validation
  ↓
Security + quality gates
  ↓
Ephemeral test environments
  ↓
Promotion across environments (dev → QA → prod)
  ↓
Observability + automated diagnosis
```

Final takeaway

The most reusable lesson is the **dependency chain**:

```
source → build artifact → container image → registry → Kubernetes workload → Se
```

When a pipeline fails, locate the broken link in that chain and inspect that layer directly.

Summary

Phase	Built	Achieved
1 — Spring Boot	REST API + actuator	Runnable microservice
2 — Unit tests	JUnit/Mockito	Fast CI feedback
3 — Docker	Multi-stage image	Portable, health-checked container
4 — Kubernetes	k3d + manifests	3 replicas, NodePort, probes
5 — API tests	REST Assured suites	Automated API validation
6 — Scripts	deploy-and-test.sh etc.	One-command operations
7 — Jenkins	Full CI/CD pipeline	GitHub → deploy → test on every build

Overall goal achieved: A complete, hands-on QA Architecture Lab demonstrating how a real team builds, containers, deploys to Kubernetes, and validates a microservice — with Jenkins automating the entire flow.

Next Steps (Roadmap)

Phase	Description
Resilience testing	Pod failure, scaling, rollback scenarios
Helm	Templatize Kubernetes manifests
Argo CD	GitOps-based deployment
Observability	Prometheus, Grafana, OpenTelemetry

See [README.md](#) for command reference and troubleshooting.